

Día D — qué ejecuta cada uno

Ramas y fases ya están repartidas. Cada quien corre **un solo comando de agente**: `/speckit-implement` con su fase. Lo único que se escribe a mano es **la Historia que dicte el profesor**.

Quién	Su rama	Su fase	Archivos que le pertenecen
Sergio	main	reparte	spec.md, tasks.md
Mateo	feature/hu-ui	fase 6	app/**/page.jsx, components/**
Johan	feature/hu-api	fase 7	app/api/**/route.js
Tomás	feature/hu-datos	fase 8	lib/**, test/**
Santiago	main	merge y deploy	—

Paso 0 · los cinco

20 minutos antes de clase

```
cd ~/Documents/web/speckit-ai-generator
git checkout main
git pull origin main
npm install
npm test # tiene que decir 0 failed
```

Santiago, además — deja el panel de Vercel abierto y la URL cargada en el celular:

```
npm run smoke https://ia-generator-
openspec.vercel.app
```

Paso 1 · Sergio mete la Historia y reparte las fases

≈ 2 min · rama **main**

```
git checkout main
git pull origin main
```

1.1 — la Historia entra al spec. Pega esto con las historias literales del profesor:

```
/speckit-specify Agrega dos User Stories nuevas al final de la seccion User Scenarios
de specs/001-ai-media-generator/spec.md, numeradas User Story 5 y User Story 6.
HU 5: "historia literal que dictó el profesor"
HU 6: "segunda historia literal"
Cada una con sus Acceptance Scenarios numerados, en formato Given/When/Then.
NO crees una feature nueva. NO corras create-new-feature.sh. NO cambies de rama.
NO toques plan.md ni tasks.md. Solo edita ese spec.md que ya existe.
```

1.2 — el reparto. Este prompt se pega tal cual, sin cambiar nada:

```
Agrega al final de specs/001-ai-media-generator/tasks.md tres fases nuevas para las
User Stories 5 y 6. No regeneres el archivo ni toques las fases 1 a 5.

## Fase 6 — HU en vivo · pantallas    Owns: app/**/page.jsx, components/**
## Fase 7 — HU en vivo · API          Owns: app/api/**/route.js
## Fase 8 — HU en vivo · datos        Owns: lib/**, test/**

Reglas: ningun archivo puede aparecer en dos fases. Numera las tareas desde T040.
Cada tarea dice que Acceptance Scenario cubre, con la etiqueta [US5-2], y es test-first:
primero la prueba, nombrada it("US5-2: ..."), despues el codigo.
Si una HU no da trabajo para alguna fase, deja esa fase con una sola tarea que lo diga.

git add specs/001-ai-media-generator/
git commit -m "HU en vivo: User Stories 5 y 6 y sus fases 6, 7 y 8"
git push origin main
```

Después dice en voz alta: «fase 6 Mateo, fase 7 Johan, fase 8 Tomás — arranquen».

Copia solo tu columna, de arriba abajo. El agente hace el resto:

MATEO

```
git checkout main
git pull origin main
git checkout -b feature/hu-ui

/speckit-implement fase 6

npm test
git add -A
git commit -m "feat: HU en vivo pantallas"
git push -u origin feature/hu-ui
```

JOHAN

```
git checkout main
git pull origin main
git checkout -b feature/hu-api

/speckit-implement fase 7

npm test
git add -A
git commit -m "feat: HU en vivo API"
git push -u origin feature/hu-api
```

TOMÁS

```
git checkout main
git pull origin main
git checkout -b feature/hu-datos

/speckit-implement fase 8

npm test
git add -A
git commit -m "feat: HU en vivo datos"
git push -u origin feature/hu-datos
```

Es `/speckit-implement`, no `/implement`, y siempre con la fase — sin argumento el agente construye todo el backlog. Si `npm test` **sale rojo, no hagas push**: dile al agente cuál prueba falla, que la arregle y vuelva a correr `npm test`. Cuando tu push termine, anúncialo en voz alta.

Paso 3 · Santiago junta las tres ramas y despliega

≈ 2 min · rama `main`

Solo cuando los tres hayan dicho que subieron:

```
git checkout main
git pull origin main
git fetch origin
git merge --no-ff origin/feature/hu-ui
git merge --no-ff origin/feature/hu-api
git merge --no-ff origin/feature/hu-datos
```

Con los tres adentro:

```
npm test
npm run build
git push origin main
```

Ese push despliega en Vercel solo. Mientras construye, proyecta el grafo de ramas:

```
git log --graph --oneline -15
```

Uno por línea, esperando a que termine cada uno. Si un merge se enreda: `git merge --abort` y reintenta solo esa rama.

Paso 4 · todos validan sobre el despliegue

≈ 1 min

1. **Santiago** espera el check verde del deploy en Vercel (1–2 min) y corre:

```
npm run smoke https://ia-generator-openspec.vercel.app
```

2. Todos abren `https://ia-generator-openspec.vercel.app` en el celular, con datos móviles, y recorren la Historia nueva de punta a punta.
3. Si preguntan de dónde salió el código, el dueño abre `tasks.md` en su fase, señala la etiqueta `[US5-2]` y corre esa sola prueba:

```
npm test -- -t "US5-2"
```